

PetClinic InfraLab

GitLab CI build (Maven) + container image push (Kaniko) with a GitLab Runner on Rancher Desktop (Kubernetes)

This guide documents the working end-to-end setup used to build and publish a Java artifact, build a container image with Kaniko, push it to the GitLab Container Registry, and (when needed) run the pipeline using a local GitLab Runner installed on Rancher Desktop.

- Maven build in CI produces a JAR and publishes it as a job artifact.
- Kaniko builds the Dockerfile and pushes the image to GitLab's registry.
- Images are tagged with the commit short SHA (CI_COMMIT_SHORT_SHA).
- When there is no suitable runner in GitLab, a local Kubernetes runner is installed to execute jobs.

Target outcome

- Pipelines start immediately (no Pending / Stuck jobs).
- Build job succeeds and exposes artifacts (JAR) in the job page.
- Image push job succeeds and the GitLab Container Registry shows an image tag equal to the commit short SHA (CI_COMMIT_SHORT_SHA).
- Registry authentication uses GitLab-provided CI variables (CI_REGISTRY, CI_REGISTRY_USER, CI_REGISTRY_PASSWORD, CI_REGISTRY_IMAGE).

Pre-flight checklist (before you click “New pipeline”)

- A CI config file is discoverable by GitLab (root **.gitlab-ci.yml**, or a root include pointing to **ci/.gitlab-ci.yml**).
- Container Registry is enabled for the project (so **CI_REGISTRY*** variables exist).
- Your jobs' tag requirements match an available runner (for example: jobs tagged **k8s** require a runner that advertises **k8s**).
- If you plan to run jobs locally, Rancher Desktop Kubernetes is running and you have Helm + kubectl installed.

Key concepts and terms

Term	What it means in this setup
Pipeline	A single run of your CI configuration on a specific commit.
Stage	A group of jobs that run in order (for example: build, then image).
Job	A single unit of work (for example: Maven build). Jobs run on a runner.
Runner	An agent that executes CI jobs. If no runner is available (or tags do not match), jobs stay Pending / Stuck.
Job tags	Labels required by a job (for example: k8s). Only runners configured with the same tag can pick up that job.
GitLab Container Registry	Per-project image registry in GitLab. Images are pushed under <code>CI_REGISTRY_IMAGE</code> .
Kaniko	A container image builder that runs without docker:dind (no privileged Docker daemon). Ideal for Kubernetes runners.
Runner token	Secret used to register/authenticate a runner with GitLab. Using the wrong token type (or a placeholder value) causes 403 Forbidden during verification.

Where to look in GitLab UI

- **Build** → **Pipelines**: see the latest pipeline and whether jobs are running or stuck.
- **Build** → **Jobs**: open job logs and download artifacts for the build job.
- **Deploy** (or **Packages & Registries**) → **Container Registry**: confirm pushed images and tags.
- **Settings** → **CI/CD** → **Runners**: confirm a runner is online and which tags it provides.

CI pipeline structure (what is implemented)

The pipeline uses two stages:

- **build**: runs Maven (JDK 17) and stores the resulting JAR(s) from **target/** as artifacts; caches **.m2/repository** to speed up future runs.
- **image**: runs Kaniko to build the Dockerfile and pushes to the project registry using a tag equal to **CI_COMMIT_SHORT_SHA**.

Build job (what to expect in logs)

- Java and Maven versions are printed first (sanity check the runner image).
- A successful **mvn clean package** completes and **target/** contains one or more JARs.
- Artifacts section in the job UI includes the JAR(s) (for example: **target/*.jar**).
- Maven cache is stored under **.m2/repository** to speed up the next run.

Image push job (Kaniko) essentials

- Kaniko reads your **Dockerfile** from the repository and builds without docker:dind/privileged mode.
- Auth is created by writing a Docker **config.json** that contains credentials for **CI_REGISTRY**.
- The push destination is **\$CI_REGISTRY_IMAGE:\$CI_COMMIT_SHORT_SHA**.
- In successful logs, you will see Kaniko building layers and a final push to the destination registry/image.

Key GitLab variables used by the image push job

- **CI_REGISTRY**: registry host (where to authenticate).
- **CI_REGISTRY_IMAGE**: image path for this project (where to push).
- **CI_REGISTRY_USER** and **CI_REGISTRY_PASSWORD**: credentials GitLab injects for registry login.
- **CI_COMMIT_SHORT_SHA**: short commit hash used as the image tag.

Critical setup: GitLab must find your CI configuration file

We hit the error “**Pipeline cannot be run. Missing CI config file**”. The fix that worked was ensuring GitLab can find the CI config at the expected path. If your main pipeline file lives under **ci/**, add a minimal root **.gitlab-ci.yml** that includes it (or configure the project setting to point to the custom path).

Minimal include example (root **.gitlab-ci.yml**)

```
include:  
  - local: "ci/.gitlab-ci.yml"
```

Step-by-step: run and validate the pipeline

1) Trigger a pipeline

- Merge to the branch that triggers CI (commonly **main**), or use **Build** → **Pipelines** → **New pipeline**.
- If GitLab shows “Missing CI config file”, fix the CI config path first (previous section).

2) Validate the build job output and artifacts

- Open **Build** → **Pipelines** and click the latest pipeline.
- Open the **build** job and confirm Maven finishes successfully.
- On the job page, confirm the **Artifacts** section shows the JAR(s) saved from **target/**.

3) Validate the image push and registry tag

- Open the **push_image** job and confirm it finishes successfully.
- In the job log, confirm the push destination contains **\$CI_REGISTRY_IMAGE:\$CI_COMMIT_SHORT_SHA**.
- Navigate to **Deploy** (or **Packages & Registries**) → **Container Registry** and confirm the short SHA tag exists.

4) Re-run after fixing runners/tokens

- If a job was stuck Pending, fix the runner first (next section).
- Then click **Retry** on the job, or run a new pipeline from **Build** → **Pipelines**.

5) If jobs are Pending / Stuck

If GitLab says there are no active runners online or no runners match the job's tags (for example: **k8s**), the CI YAML is not the problem. It means there is no runner available that can execute the jobs.

Step-by-step: install a GitLab Runner on Rancher Desktop (Kubernetes)

Use this approach when GitLab has no suitable runner available (jobs stay Pending / Stuck).

Prerequisites

- Rancher Desktop installed with Kubernetes enabled.
- kubectl configured to point to the Rancher Desktop cluster context.
- Helm installed on your machine.
- A valid GitLab runner authentication token from your GitLab project/group (next section).

1) Confirm kubectl is pointing to the right cluster

```
kubectl cluster-info
kubectl get nodes
kubectl config current-context
```

2) Create the namespace (fix for: namespaces "gitlab-runner" not found)

```
kubectl create namespace gitlab-runner
```

3) Add the GitLab Helm repo

```
helm repo add gitlab https://charts.gitlab.io
helm repo update
```

4) Install/upgrade the runner using your values file

We installed the Helm chart into the **gitlab-runner** namespace using a values file. The values file must include: the GitLab URL, the runner token, and runner tags (for example: **k8s**) that match your CI jobs.

Shell-agnostic (single line):

```
helm upgrade --install gitlab-runner gitlab/gitlab-runner -n gitlab-runner -f gitlab-runner.yml
```

PowerShell (multi-line with backticks):

```
helm upgrade --install gitlab-runner gitlab/gitlab-runner `
-n gitlab-runner `
-f gitlab-runner.yml
```

5) Verify the runner pod and inspect logs if not Ready

```
kubectl get pods -n gitlab-runner
kubectl logs -n gitlab-runner deploy/gitlab-runner --tail=200
```

Expected healthy state

- Pod shows **READY 1/1** and does not restart repeatedly.
- In GitLab UI (Settings → CI/CD → Runners), the runner appears as **online** with the expected tag(s).
- When the runner is online, jobs tagged **k8s** stop being stuck and start running.

Runner token: what it is, where to get it, and the bug we hit

A runner token is the credential that allows a runner to authenticate to GitLab and register itself. Without a valid token, GitLab rejects the runner during verification and the runner never becomes online.

Where to get the token (GitLab UI)

- Open the project in GitLab.
- Go to **Settings** → **CI/CD** and expand **Runners**.
- Create a **New project runner** (or use a group runner if that's the intended scope).
- Copy the **runner authentication token** that GitLab provides.

What went wrong (and how we detected it)

After installing the Helm chart, the runner pod was **Running** but **not Ready (0/1)** and kept restarting. Runner logs showed repeated verification attempts ending with:

```
POST https://gitlab.endava.com/api/v4/runners/verify: 403 Forbidden
```

Fix that works for this 403 pattern

- Use the correct **runner authentication token** (new runner flow) from GitLab. On many GitLab instances this token format begins with **glrt-**.
- Ensure the token is passed into the Helm values as a real value, not a placeholder like **\${SOME_VAR}**.
- After updating the values file, run **helm upgrade** again and re-check the pod logs until the runner becomes online.

Token placeholder bug (what it looked like)

In the logs, we saw the runner field printed as an incomplete environment reference (for example: **runner=\${GITLAB_}**). That's a sign the values file contained an unexpanded placeholder instead of the real token. Replacing it with the actual token value resolved this class of failure.

Troubleshooting (errors we hit and the fixes that worked)

Symptom / error	Cause	Fix that worked
Pipeline cannot be run. Missing CI config file	GitLab could not locate the CI config file at the expected path.	Ensure a root .gitlab-ci.yml exists, or add a root include that points to ci/.gitlab-ci.yml (or configure the project CI/CD setting to the correct path).
Jobs Pending / Stuck; message mentions no runners or no runners match tags: k8s	No runner is online with matching tags (or runner not allowed for the branch).	Install/register a runner with the required tag(s) (k8s) and confirm it appears Online in GitLab. Then re-run the pipeline.
helm upgrade --install ... Error: namespaces "gitlab-runner" not found	Namespace does not exist yet.	Create the namespace first: <code>kubectl create namespace gitlab-runner</code> , then rerun Helm.
Runner pod Running but not Ready (0/1); logs show verify attempts and 403 Forbidden	Runner is not authenticated/registered (wrong token type or token not injected correctly).	Create a new runner in GitLab and use its runner authentication token (often glrt-...). Ensure the token is a real value in the Helm values file (not <code>\${VAR}</code>), then helm upgrade again.

Acceptance checklist

- Runner shows as **Online** in GitLab (Settings → CI/CD → Runners) and provides the required tag(s) (for example: k8s).
- A pipeline run completes with no jobs stuck in Pending.
- Build job succeeds and artifacts are downloadable (JAR from target/).
- Image push job succeeds and the Container Registry shows an image tag equal to the commit short SHA.

Files you will typically attach alongside this guide

- Root .gitlab-ci.yml (minimal include if the main config lives under ci/).
- ci/.gitlab-ci.yml (pipeline jobs and stages).
- gitlab-runner values file used for Helm (GitLab URL, runner token, runner tags).